Processor Design Project (PDP)

CESE4040 (Q4 2025-2026)

Compiler Workflow Manual

Author:

Mahmood Naderan-Tahan

Table of Contents

1. Introduction

This document is the second part of the PDP course (CESE4040) workflow in Q4, 2024-2025. While the first part deals with modifying a RISC-V core to support hardware AES instruction support, the second part focuses on the back-end side of the compiler for further optimizations. Specifically, the goal is to 1) use an existing LLVM optimization pass in the first milestone and 2) write a back-end optimization pass as plugin to LLVM infrastructure.

2. Implementing AES instructions

You are given a software implementation of AES encryption algorithm in C. The function which we focus is shown below consisting three steps: 1) Initial round, 2) Middle rounds with 9 iterations and 3) Final round.

```
// Single block AES-128 encryption
void aes128_encrypt_block(uint8_t *plaintext, uint8_t *round_keys,
                          uint8_t *ciphertext) {
    …

    // Initial round
    add_round_key(state, round_keys);

    // Main rounds (1 to 9)
    for (int round = 1; round < 10; round++) {
        sub_bytes(state);
        shift_rows(state);
        mix_columns(state);
        add_round_key(state, &round_keys[round * 16]);
    }

    // Final round (10)
    sub_bytes(state);
    shift_rows(state);
    add_round_key(state, &round_keys[10 * 16]);

    …
}
```

To speedup the implementation on RISC-V architecture, custom instructions are defined in ZKNE extension. Here, we focus on the middle and final rounds, to implement `aes32esmi` and `aes32esi` instructions, respectively. In the first part of the course, before the milestone, you are required to:

1) Modify the C code to use `aes32esmi` and `aes32esi`.
2) Obtain the IR representation of the code and verify that assembly instructions are used properly.

3) Apply the LLVM “loop-unroll” optimization pass to the modified code and verify that the loop has been unrolled.

The description of the instructions can be found in ZKNE manual. The manual explains how the instructions work and how they should be used. Since the description and general usage of `aes32esmi` and `aes32esi` are similar, we only explain the details of the first instruction. The format of `aes32esmi` looks like:

```
aes32esmi rd, rs1, rs2, bs
```

Where

- `rd`: Destination register
- `rs1` and `rs2`: Source registers
- `bs`: Byte shift value

To be able to use the assembly instruction in the C code, `__asm__` directive is used as shown below:

```
__asm__ volatile (
       "aes32esmi %0, %1, %2, %3"
         : "+r" (X1)
         : "r" (X2),
           "r" (X3),
           "I" (X4)
);
```

Where `%0`, `%1`, `%2`, and `%3` are corresponding to `rd`, `rs1`, `rs2`, and `bs`, respectively. Please note that you have to complete the instruction by properly filling X1..X4.

To verify that the modified code, you can quickly compile and execute the code and compare the output (encrypted text) with the software implementation code. Since the input and key are hardcoded in the source file, it is expected to get a single unique output. A sample is shown below:

2. Standard “loop-unroll”

Once you have verified that the assembly instructions are functionally correct, you can proceed with LLVM optimization passes. Here, we focus on the “loop-unroll” pass on the initial IR representation, so the effect would be the unrolled middle round of AES encryption. While there are several passes overlapping functions, it is mandatory to apply “loop-unroll” and get the final correct result. To check if the loop has been unrolled, you should first count the number of

`aes32esmi` instructions inside the original loop and then multiply that by the number of iterations, fixed 9 times. The same result must be observed in the unrolled version.

3. Custom LLVM Optimization Pass

In the second part of the course, after the milestone, you are asked to implement a simple loop unroll pass and add it to the LLVM infrastructure. To do that, we recommend to:

1. Understand how an loop unrolling works by reading online pages. You can read this document for more information.
2. Understand how to define a custom pass. You can start with a "HelloWorld" pass from LLVM manual. You can define a pass via a shared library plugin or define it in the LLVM source tree.
3. Get familiar with LLVM internals, e.g. modules, basic block data type. See this tutorial and this manual for more information.
4. Write a simple loop-unroll pass which is functional with the AES middle round lop.